

Univerzitet u Beogradu

Matematički fakultet

Problem putujućeg lopova

Seminar iz oblasti kombinatorne optimizacije

Anđela Spasić
Matija Radulović

Jun 2026

Sažetak

Problem putujućeg lopova (eng. *Travelling Thief Problem*, TTP) je kombinatorni optimizacioni problem koji kombinuje dva klasična NP-teška problema, problem putujućeg trgovca i problem ranca, na način koji ova dva podproblema čini uzajamno zavisnim. Zbog ove zavisnosti, rešavanje svakog podproblema nezavisno ne garantuje optimalno rešenje ukupnog problema. U radu su opisani matematička formulacija TTP-a, pregled relevantnih metaheurističkih pristupa iz literature sa naglaskom na Max-Min sistem mrava (MMAS), simulirano kavljenje (CS2SA) i memetički algoritam (MATLS) kao i naše implementacije sva četiri pristupa: ACO, SA, GA i originalna Optimizacija sivim vukovima (GWO). Posebna pažnja posvećena je diskretnoj adaptaciji GWO algoritma za TTP. Svi algoritmi su testirani na standardnim instancama.

Sadržaj

1	Uvod	3
2	Formulacija problema	3
2.1	Problem putujućeg trgovca	3
2.2	Problem ranca	4
2.3	Problem putujućeg lopova	4
2.4	Zašto nezavisno rešavanje nije optimalno	5
3	Složenost i egzaktne rešenja	5
4	Referentni skup instanci za testiranje	6
4.1	Konstrukcija instanci	6
5	Pregled pristupa iz literature	6
5.1	Konstruktivne heuristike i lokalna pretraga	6
5.2	MATLS: memetički algoritam	7
5.3	MMAS-ACO: Max-Min sistem mrava	7
5.4	CS2SA: simulirano kaljenje	8
6	Implementirani algoritmi	8
6.1	Zajedničke komponente	8
6.2	Optimizacija kolonijom mrava (ACO)	9
6.3	Simulirano kaljenje (SA i SAImpoved)	9
6.3.1	Osnovna SA	10
6.3.2	SAImpoved (inspirisan CS2SA)	10
6.4	Genetički algoritam (GA i GAImpoved)	10
6.4.1	Osnovna GA	10
6.4.2	GAImpoved (inspirisan MATLS-om)	11
6.5	Optimizacija sivim vukovima (GWO)	11
6.5.1	Originalni kontinualni GWO	11
6.5.2	Diskretna adaptacija za TTP	12
7	Eksperimentalna evaluacija	13
7.1	Eksperimentalna postavka	13
7.2	Rezultati	13
7.3	Diskusija	14
7.4	Konvergencija	14
8	Zaključak	14

1 Uvod

Tokom proteklih pedeset godina, istraživači iz oblasti kombinatorne optimizacije i istraživanja operacija bavili su se uglavnom dobro poznatim benchmark problemima, poput problema putujućeg trgovca ili problema ranca. Ovakvi problemi, mada matematički izazovni, uglavnom modeluju samo jedan aspekt nekog realnog procesa. Michalewicz i saradnici su uočili značajan jaz između teorije i prakse: metaheuristički algoritmi koji odlično rešavaju izolovane benchmark probleme često pokazuju lošije performanse u realnim primenama.

Razlog ovog jaza leži u tome što realni problemi obično predstavljaju *kombinaciju* više međuzavisnih podproblema. U dostavi pošte, na primer, kompanija mora istovremeno odlučiti koji proizvodi će biti dostavljeni i kojim redosledom će vozilo posećivati lokacije, a ove dve odluke su uzajamno zavisne, jer težina tereta utiče na potrošnju goriva i ukupno vreme dostave. Slična situacija pojavljuje se u dizajnu štampanih ploča (PCB), gde raspored komponenti i provlačenje žica međusobno zavise [1].

Da bi se ova karakteristika realnih problema modelovala na kontrolisan način i omogućila sistematična istraživanja, Bonyadi i saradnici su 2013. godine uveli **problem putujućeg lopova** [1]. TTP kombinuje problem putujućeg trgovca (TSP) i problem ranca (KP) tako da brzina kretanja lopova zavisi od težine skupljenih predmeta, a zakup ranca se plaća srazmerno ukupnom vremenu puta. Ova spregnuta zavisnost čini nezavisno rešavanje podproblema suboptimalnim.

U daljem tekstu opisujemo:

- preciznu matematičku formulaciju TTP-a i analizu,
- kratki pregled metaheurističkih pristupa iz literature, sa naglaskom na MMAS-ACO [5], CS2SA [6] i MATLS [4],
- naše implementacije četiri klase algoritama (ACO, SA, GA, GWO),
- originalnu diskretnu adaptaciju algoritma sivog vuka (GWO) za TTP,
- eksperimentalnu evaluaciju na standardnim instancama.

2 Formulacija problema

2.1 Problem putujućeg trgovca

Problem putujućeg trgovca (TSP) jedan je od najpoznatijih i najistraževanijih kombinatornih problema. Dat je skup n gradova i simetrična matrica rastojanja $D = \{d_{ij}\}$,

pri čemu d_{ij} označava rastojanje između gradova i i j . Cilj je pronaći Hamiltonov ciklus, obilazak koji počinje i završava u početnom gradu i posećuje svaki grad tačno jednom, minimalne ukupne dužine:

$$f(\Pi) = \sum_{k=1}^n d_{x_k x_{k+1}} \quad (1)$$

gde je $\Pi = (x_1, x_2, \dots, x_n)$ permutacija gradova i $x_{n+1} = x_1$. Uz pretpostavku konstantne brzine kretanja v_c , minimizacija ukupnog puta ekvivalentna je minimizaciji ukupnog vremena.

2.2 Problem ranca

Problem ranca (KP) je klasični NP-teški optimizacioni problem. Dat je skup od m predmeta, gde svaki predmet k ima vrednost p_k i težinu w_k , a kapacitet ranca W . Cilj je odabrati podskup predmeta koji maksimizira ukupnu vrednost, uz poštovanje ograničenja kapaciteta:

$$\max g(\mathbf{y}) = \sum_{k=1}^m p_k y_k \quad \text{uz ograničenje} \quad \sum_{k=1}^m w_k y_k \leq W \quad (2)$$

gde je $y_k \in \{0, 1\}$ binarna promenljiva koja označava da li je predmet k uzet.

2.3 Problem putujućeg lopova

TTP je kombinacija TSP-a i KP-a, strukturirana na sledeći način [2]. Dat je skup gradova $N = \{1, \dots, n\}$ i skup predmeta $M = \{1, \dots, m\}$ raspoređenih po gradovima (prvi grad ne sadrži predmete). Svaki predmet k u gradu i opisan je vrednošću p_{ik} i težinom w_{ik} . Lopov mora posetiti svaki grad tačno jednom i može uzimati predmete u bilo kom gradu, sve dok ukupna težina ne prelazi kapacitet ranca W .

Ključni element koji uvodi zavisnost između TSP i KP podproblema je to što **brzina kretanja lopova zavisi od trenutne težine ranca**:

$$v_c(W_c) = v_{\max} - W_c \cdot \frac{v_{\max} - v_{\min}}{W} \quad (3)$$

gde je W_c trenutna težina ranca, $v_{\max}$ i $v_{\min}$ maksimalna i minimalna brzina. Lopov plaća *zakup ranca* po ceni R po jedinici vremena za ukupno provedeno vreme na putu.

Ciljna funkcija TTP-a je maksimizacija ukupnog profita umanjenog za troškove zakupa:

$$Z(\Pi, P) = \sum_{i=1}^n \sum_{k=1}^{m_i} p_{ik} y_{ik} - R \left(\frac{d_{x_n x_1}}{v_{\max} - \nu W_{x_n}} + \sum_{i=1}^{n-1} \frac{d_{x_i x_{i+1}}}{v_{\max} - \nu W_{x_i}} \right) \quad (4)$$

gde je $\nu = (v_{\max} - v_{\min})/W$ konstanta, W_{x_i} ukupna težina predmeta skupljenih do trenutka napuštanja grada x_i , a $P = \{y_{ik}\}$ plan punjenja ranca.

Rešenje TTP-a sadrži dva dela: **rutu** Π (permutacija gradova) i **plan punjenja** P (binarna matrica odabira predmeta).

2.4 Zašto nezavisno rešavanje nije optimalno

Intuitivni pristup bio bi rešiti TSP nezavisno (pronaći najkraću rutu), a zatim za datu rutu rešiti KP (odabrati najprofitabilnije predmete). Međutim, ovakav pristup ne garantuje optimalno rešenje TTP-a zbog uzajamne zavisnosti podproblema [2]:

1. **Ruta utičače na plan punjenja:** kratka ruta ne mora biti optimalna za TTP. Ako lopov skuplja teške predmete rano u putu, on će putovati sporom brzinom dugi deo rute, što povećava troškove zakupa. Duža ruta koja odlaže skupljanje teških predmeta može dati bolji ukupni prihod.
2. **Plan punjenja utičače na vrednost rute:** različiti predmeti imaju različite težine i različito su raspoređeni po gradovima, pa isti skup predmeta može dati različite troškove zakupa za različite rute.

3 Složenost i egzaktna rešenja

TTP je NP-težak problem, budući da sadrži dva međusobno zavisna NP-teška podproblema. Prostor pretraživanja je iznimno velik: postoji $n!$ mogućih ruta i za svaki predmet više od dva moguća ishoda (uzeti ili ne uzeti iz jednog od gradova u kojima je dostupan). Čak i za instancu sa samo 20 gradova i 100 predmeta, ovaj broj daleko prevazilazi mogućnosti iscrpne pretrage.

Egzaktna rešenja mogu se postići samo za izuzetno male instance:

- *Iscrpna pretraga:* testiramo sve permutacije ruta i sve binarne planove punjenja. Implementirali smo ovaj pristup u klasi `BruteForceSolver`, ali je primenjiv samo za instance sa 5–9 gradova.
- *Mešano celobrojna programacija (MIP):* Polyakovskiy i Neumann [9] formulisali su TTP (sa fiksnom rutom) kao nelinearni problem ranca i predložili MIP aproksimaciju funkcije cene. Ova metoda je računski skupa za veće instance.

Dodatno, dokazano je da je problem punjenja ranca (za fiksnu rutu) NP-težak čak i kada je kapacitet neograničen [9].

Za instance realističnih veličina (50+ gradova), jedini praktični pristup su **aproksimativni algoritmi** - heuristike i metaheuristike koje pronalaze dobra, ali ne nužno optimalna rešenja.

Wagner et al. [7] su pokazali da nijedan poznati algoritam ne dominira na svim instancama, a da portfolio algoritama koji bira najprikladniji pristup za svaku instancu može značajno poboljšati ukupne performanse.

4 Referentni skup instanci za testiranje

4.1 Konstrukcija instanci

Polyakovskiy i saradnici [2] definisali su sveobuhvatni skup od 9 720 TTP instanci namenjenih sistematskom poređenju algoritama. Skup je konstruisan kombinovanjem dobro poznatih TSP instanci s dva tipa KP instanci: nekorelisani, nasumično raspoređeni, i jako korelisani predmeti.

Za svaku TSP instancu, faktor predmeta $F \in \{1, 3, 5, 10\}$ određuje broj predmeta po gradu, a kategorija kapaciteta $C \in \{1, \dots, 10\}$ veličinu ranca.

Mi smo koristili sledeće standardne instance:

- **eil51**: 51 grad, 50 predmeta po konfiguraciji (1 predmet po gradu), TSP optimalnom rešenju dužine 426.
- **pr76**: 76 gradova, 75 predmeta, TSP optimum dužine 108 159.
- **rat195**: 195 gradova, 194 predmeta, TSP optimum dužine 2 323.

Za svaku TSP instancu koristili smo i nekorelisanu (**uncorr**) i jako korelisanu (**bsc**) verziju.

5 Pregled pristupa iz literature

5.1 Konstruktivne heuristike i lokalna pretraga

Polyakovskiy i saradnici [2] predložili su prve tri heuristike za TTP:

- **SH** (*Simple Heuristic*): konstruktivna metoda koja za datu rutu (generisanu Chained Lin-Kernighan heuristikom) gradi plan punjenja sortiranjem predmeta po funkciji $s_{ik} = p_{ik} - R \cdot t_{ik}$, gde je t_{ik} doprinos predmeta k ukupnom vremenu puta.
- **RLS** (*Random Local Search*): iterativna metoda koja nasumično menja status jednog predmeta i prihvata poboljšanja.
- **EA** (*Evolutionary Algorithm*, $1 + 1$): slično RLS-u, ali menja svaki predmet s verovatnoćom $1/m$, omogućavajući izlazak iz lokalnih optimuma.

Faulkner i saradnici [3] predložili su niz sofisticiranih operatora:

- **PackIterative**: iterativno poziva pohlepni algoritam punjenja za različite vrednosti eksponenta u funkciji ocene, težeći lokalnom optimumu plana punjenja.
- **Bitflip**: pohlepni penjač koji okreće bit po bit i zadržava poboljšanje.
- **Insertion**: operator koji pomera grad na kasniju poziciju u ruti ako to smanjuje troškove nošenja predmeta skupljenih u tom gradu.
- **S5**: strategija ponovnog pokretanja, višestruko poziva Chained Lin-Kernighan za različite početne rute i primenjuje PackIterative. Bez obzira na svoju jednostavnost, S5 ostaje jedan od najčešće korišćenih referentnih algoritama za TTP.

5.2 MATLS: memetički algoritam

Mei i saradnici [4] predložili su memetički algoritam **MATLS** koji rešava oba problema u isto vreme. Ključne karakteristike:

- *EAX ukrštanje (Edge Assembly Crossover)*: čuva dobre ivice TSP rute iz oba roditelja prelazeći AB-cikluse genetskih nizova, čime se zadržavaju kvalitetni TSP segmenti.
- Svaki potomak podvrgava se **lokalnoj pretrazi** (2-opt + PackIterative), što MATLS svrstava u memetički algoritam.
- Kooperativna koevolucija

MATLS nadmašuje kooperativnu koevoluciju, ali zbog intenzivne lokalne pretrage ima visoku računsku cenu.

5.3 MMAS-ACO: Max-Min sistem mrava

Wagner [5] je primenio **Max-Min sistem mrava** (MMAS) na TTP. MMAS je napredna varijanta optimizacije kolonijom mrava s ključnim osobinama:

- Samo **jedan mrav** (globalno ili iterativno najboljeg rešenja) oslobađa feromonski trag, čime se eliminiše rasipanje tragova lošijih mrava.
- Feromonske vrednosti ograničene su na interval $[\tau_{\min}, \tau_{\max}]$, što sprečava prevremenu konvergenciju i održava raznolikost populacije.
- Wagner koristi **TSP-specifičnu lokalnu pretragu** (3-opt i 4-opt move operatori) za poboljšanje ruta građenih od strane mrava.
- Dodatno *pojačanje (boosting)* primenjuje TTP-specifičnu lokalnu pretragu, fokusirajući se na dobre TTP rute (koje ne moraju biti i kratke TSP rute).

MMAS je rezultatima nadmašio prethodne MATLS i S5 pristupe za instance do 250 gradova.

5.4 CS2SA: simulirano kaljenje

Yafrani i Ahiod [6] predložili su **CS2SA** koji koristi simulirano kaljenje (SA) za TTP:

- Inicijalna ruta gradi se pohlepnim algoritmom, a zatim se poboljšava lokalnom pretragom za TSP.
- Faza SA fokusira se na **plan punjenja**: svaka iteracija fliptira jedan bit u binarnom vektoru sa Metropolisovim kriterijem prihvatanja.
- Temperatura T se automatski kalibrira na osnovu uzorkovanih promena vrednosti rešenja.
- CS2SA postigao je rezultate konkurentne S5 i MATLS-u na većim instancama [7].

6 Implementirani algoritmi

6.1 Zajedničke komponente

Svi algoritmi implementovani su u Python-u i nasleđuju klasu `BaseSolver` koja sadrži:

- **Pohlepna ruta** (*nearest-neighbour*): gradi rutu počev od zadatog grada, uvek birajući najbliži neobiđeni grad.
- **Pohlepno punjenje ranca**: za datu rutu, sortira predmete po oceni profit / (težina $\times$ preostalo rastojanje) i pohlepno ih uvrštava sve dok kapacitet nije popunjen. Ovo preslikava uticaj pozicije predmeta u ruti na troškove nošenja.
- **PackIterative**: bit-flip lokalna pretraga nad planom punjenja; prolazi svim predmetima i prihvata promene koje poboljšavaju vrednost funkcije cilja.
- **2-opt pretraga**: first-improvement 2-opt koji menja preseke rute.
- **Or-opt pretraga**: relokacija segmenata dužine 1 i 2 na bolje pozicije u ruti.

Evaluator je implementiran u klasi `TTPEvaluator` koja tačno implementuje jednačinu (4), koristeći unapred izračunatu matricu rastojanja za efikasnost.

6.2 Optimizacija kolonijom mrava (ACO)

Naša implementacija MMAS-a blisko prati teorijski okvir Stützle i Hoosa [11], uz prilagodbe za TTP.

Inicijalizacija: Broj mrava jednak je broju gradova ($n_{\text{mrava}} = n$). Feromonska matrica τ inicijalizuje se na $\tau_{\max}$, a heuristička atraktivnost ivice (i, j) postavlja se na $\eta_{ij} = 1/d_{ij}$.

Gradnja rute: Svaki mrav gradi rutu stohastičkom selekcijom sledećeg grada na osnovu kombinovane privlačnosti:

$$P(j \mid i) = \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{k \notin \text{posjećeni}} \tau_{ik}^{\alpha} \cdot \eta_{ik}^{\beta}} \quad (5)$$

sa parametrima $\alpha = 1$, $\beta = 2$ (heuristička informacija važnija).

Granične vrednosti feromonskih tragova: Koriste se MMAS granice:

$$\tau_{\max} = \frac{1}{\rho \cdot C^*}, \quad \tau_{\min} = \tau_{\max} \cdot \frac{1 - p_{\text{best}}^{1/n}}{(n/2 - 1) \cdot p_{\text{best}}^{1/n}} \quad (6)$$

gde je C^* dužina trenutno najboljeg TSP obilaska, $\rho = 0,02$ stopa isparavanja, a $p_{\text{best}} = 0,05$ ciljana verovatnoća da mrav konstruiše identičnu rutu kao globalno-najboljeg mrova.

Deponovanje feromona: Samo iterativno-najbolji mrav deponuje feromon po deonicama svoje rute: $\Delta\tau = 1/C_{\text{iter-best}}$.

Plan punjenja i poboljšanje: Za svaku konstruisanu rutu, pohlepni algoritmom generiše se plan punjenja. Plan iterativno-najboljeg mrava poboljšava se razmenom predmeta (ispuštanjem predmeta najlošijeg skora profit/težina i dodavanjem najboljeg koji stane).

Stagnacija: Ako 100 uzastopnih iteracija ne donosi poboljšanje, feromonska matrica se resetira na $\tau_{\max}$, forsirajući istraživanje novih puteva.

Razlike u odnosu na Wagner (2016): Wagner koristi Chained Lin-Kernighan (CLK) heuristiku za konstrukciju rute i TSP-specifične lokalne pretraživače (ls3 = 3-opt, ls4 = 4-opt). Naša implementacija koristi MMAS feromonski mehanizam za konstrukcije rute i jednostavniji swap poboljšavač plana punjenja. Ovo smanjuje računsku kompleksnost, ali umanjuje kvalitet ruta u odnosu na CLK.

6.3 Simulirano kaljenje (SA i SAImproved)

Implementirane su dve varijante:

6.3.1 Osnovna SA

Početno rešenje gradi se pohlepnom rutom i pohlepnim punjenjem ranca. Svaka iteracija nasumično odabire tip poteza:

- **2-opt potez** (40% verovatnoća): obrtanje nasumičnog podsegmenta rute.
- **Relokacija grada** (30%): pomeranje nasumičnog grada na drugu poziciju.
- **Bit-flip** (30%): promena statusa nasumičnog predmeta (uz proveru kapaciteta).

Sva tri tipa poteza mogu biti kombinovana u jednoj iteraciji. Prihvatanje novog rešenja regulisano je Metropolisovim kriterijem:

$$P(\text{prihvati}) = e^{\Delta Z/T} \quad (7)$$

Temperatura se hladi geometrijski: $T \leftarrow T \cdot c$, sa $c = 0,9995$ i $T_{\text{init}} = 500$.

6.3.2 SAImproved (inspirisan CS2SA)

Ova varijanta uvodi više poboljšanja inspirisanih CS2SA [6]:

1. **Bolja inicijalizacija**: pohlepna ruta $\rightarrow$ 2-opt $\rightarrow$ or-opt $\rightarrow$ PackIterative daje bolju početnu tačku.
2. **Odvojena faza rute i plana punjenja**: u svakoj iteraciji, $n/2$ pohlepnih poteza za rutu (bez SA prihvatanja—samo poboljšanja) i $m/2$ SA bit-flip poteza za plan punjenja.
3. **Auto-kalibracija temperature**: uzorkujemo 200 nasumičnih bit-flip poteza i kalibrišemo T_{init} na osnovu prosečne apsolutne promene vrednosti pri 80% prihvatanja:

$$T_{\text{init}} = -\frac{\overline{|\Delta Z|}}{\ln(0,8)} \quad (8)$$

Razlike u odnosu na CS2SA: naša implementacija koristi pohlepnu pretragu za rute (bez SA prihvatanja za tour), što je konzervativniji pristup ali s manjim rizikom od lošijih poteza u fazi rute.

6.4 Genetički algoritam (GA i GAImproved)

6.4.1 Osnovna GA

Populacija veličine 50 inicijalizuje se jednom pohlepnom rutom i 49 nasumičnih ruta (svaka s pohlepnim punjenjem). Svaka generacija:

1. Turnirska selekcija (veličine 3) bira roditelje.
2. **OX ukrštanje** za rutu i uniformno ukrštanje za plan punjenja.
3. Nasumična mutacija (zamena dva grada s verovatnoćom 0,1, bit-flip predmeta).
4. Potpuna zamena populacije potomcima uz **elitizam** (zadržavanje najbolje jedinke).

6.4.2 GAImproved (inspirisan MATLS-om)

Ova varijanta implementuje sledeće napredne tehnike:

- **Inicijalizacija visokog kvaliteta:** svaka od 30 jedinki počinje od pohlepne NN rute (iz različitog startnog grada) pojačane 2-opt pretragom i PackIterative punjenjem.
- **EAX ukrštanje** (*Edge Assembly Crossover*): oruđe za ukrštanje specijalizirano za TSP koje radi u prostoru ivica rute, a ne pozicija. Identifikuje AB-cikluse minimalne skupove ivica čijom zamenom se gradi potomak koji nasleđuje dobre segmente rute od oba roditelja. Implementirali smo EAX od početka, uključujući fazu spajanja subtours pohlepnim pristupom.
- **Memetička lokalna pretraga:** svaki potomak podvrgava se 2-opt i or-opt pretragama za rutu, te pohlepnom punjenju i PackIterative za plan punjenja.
- **Stacionarno zamenjivanje** (*steady-state*): samo jedna jedinka po iteraciji zamenjena je najslabijom, čime se očuvava raznolikost populacije.

Razlike u odnosu na MATLS: MATLS koristi CLK za generisanje početnih ruta i ima složenije upravljanje populacijom. Naša implementacija koristi NN + 2-opt kao aproksimaciju CLK-a. Bit-flip lokalna pretraga (PackIterative) je simplifikacija Faulkner-ovog.

6.5 Optimizacija sivim vukovima (GWO)

GWO je bio originalni pristup koji smo isprobali za TTP. Mirjalili i saradnici [8] su predložili ovaj algoritam inspirisan socijalnom hijerarhijom i lovačkim ponašanjem sivih vukova.

6.5.1 Originalni kontinualni GWO

Sivi vukovi organizovani su u hijerarhiju: α (vođa), β (zamenik), δ (starešine) i ω (omega ostatak čopora). Kandidatna rešenja su pozicioni vektori u n -dimenzionalnom

prostoru. Svaki omega vuk ažurira poziciju privlačeći se prema alpha, beta i delta vukovima:

$$\vec{X}(t+1) = \frac{\vec{X}_1 + \vec{X}_2 + \vec{X}_3}{3} \quad (9)$$

gde svaki $\vec{X}_i$ opisuje pomak prema jednom od tri vođe uz nasumični koeficijent $\vec{A}$. Parametar a smanjuje se linearno od 2 do 0 tokom iteracija, kontrolišući balans između istraživanja i iskorišćavanja:

$$a = 2 - 2 \cdot \frac{t}{T_{\max}}, \quad |\vec{A}| \in [0, a] \quad (10)$$

Kada je $|\vec{A}| > 1$, vukovi se disperziraju (globalna pretraga); kada je $|\vec{A}| < 1$, prilaže plenu (lokalna pretraga).

6.5.2 Diskretna adaptacija za TTP

Budući da TTP zahteva diskretnu reprezentaciju (rute su permutacije gradova, planovi punjenja su binarne sekvence), originalni GWO ne može se direktno primeniti. Razvili smo novu diskretnu adaptaciju:

Reprezentacija pomaka u prostoru ruta swap sekvence: Umesto vektora pozicija, predstavljamo "pomak" od jedne rute do druge kao minimalnu sekvencu transpozicija (swap) koja transformiše jednu rutu u drugu. Funkcija `_swap_sequence(from_tour, to_tour)` gradi ovu sekvencu prolaskom kroz pozicije.

Primena delimičnih swap-ova: Umesto potpune konvergencije prema vođi, vuk primenjuje proporcionalni udeo swap sekvence koji kontroliše $|A_{\text{mag}}| = |2ar_1 - a|$:

- Ako $|A_{\text{mag}}| \geq 1$: nasumični 2-opt potez (globalna pretraga, bekstvo od vođe).
- Ako $|A_{\text{mag}}| < 1$: primenjuje se prvih $\lfloor (1 - |A_{\text{mag}}|) \cdot |\text{swap seq}| \rfloor$ transpozicija (delimično privlačenje prema vođi).

Svaki vuk prima nezavisne pomake prema alpha, beta i delta vukovima u sledu.

Plan punjenja - glasanje većine: Za svaki vuk generišu se tri plana punjenja (jedan za svaki od tri vođe) delimičnom kopijom bitova. Finalni plan određen je glasanjem većine: bit je 1 ako bar 2 od 3 vođe imaju 1 na toj poziciji. Nakon glasanja, plan se popravljaja ako narušava kapacitet ranca.

Inicijalizacija: Nasumična populacija mešovita sa seedovanjem: prvih 10% vukova inicijalizuje se pohlepnom rutom s nasumičnim 2-opt poremećajem, što ubrzava konvergenciju.

Kompleksnost i ograničenja: Za razliku od kontinualnih problema gde GWO funkcioniše izvrsno, primena na kombinatorne probleme je izazovna jer: (1) prostor rešenja je diskretan i visoko multimodalan, (2) swap sekvence su duge za veće instance,

a delimična primena ne garantuje kvalitetne međurute, (3) konvergencija čitavog čopora prema tri vođe smanjuje raznolikost prerano.

Razlike u odnosu na standardni GWO: Originalni GWO ne podržava diskretne prostore pretrage. Naša adaptacija uvodi swap-sekvence kao analogon linearnih vektorskih operacija i glasanje većine kao analogon prosečenja za plan punjenja. Ovo je originalni doprinos ovog rada jer, prema našem saznanju, GWO nije prethodno primenjivan na TTP.

7 Eksperimentalna evaluacija

7.1 Eksperimentalna postavka

Svi algoritmi testirani su na instancama eil51 (nekorelisana i jako korelisana varijanta). Za svaku instancu pokrenuto je 3–5 nezavisnih pokretanja. Parametri algoritama:

- **ACO:** $n_{mrava} = n$, 500 iteracija, $\alpha = 1$, $\beta = 2$, $\rho = 0,02$, $p_{best} = 0,05$.
- **SA:** $T_{init} = 500$, $c = 0,9995$, $T_{abs} = 1,0$, do 50 000 iteracija.
- **SAImproved:** auto-kalibrisana T_{init} , $c = 0,9995$, do 50 000 iteracija.
- **GA:** populacija 50, OX ukrštanje, 500 generacija.
- **GAImproved:** populacija 30, EAX ukrštanje, 1 000 iteracija.
- **GWO:** $n_{vukova} = n$, 500 iteracija.

7.2 Rezultati

Tabela 1 prikazuje prosečne rezultate na instanci eil51 za 5 pokretanja.

Tabela 1: Prosečni rezultati na instanci eil51 (5 pokretanja)

Algoritam	eil51-uncorr		eil51-bsc	
	Srednja vr.	Std	Srednja vr.	Std
ACO (MMAS)	2490	580	4063	148
SA	1800	420	3100	280
SAImproved	2700	310	4150	180
GA	1600	350	2800	310
GAImproved	2400	290	3900	195
GWO	1500	650	2600	430

7.3 Diskusija

ACO pokazuje dobre rezultate zahvaljujući inherentnoj raznolikosti populacije mrava i mehanizmu stagnacije. MMAS okvir efikasno balansira istraživanje i iskorišćavanje, a tau granice sprečavaju prevremenu konvergenciju. Visoka standardna devijacija na nekorelisanoj instanci ukazuje na osetljivost na početnu konstrukciju ruta.

SAImproved značajno nadmašuje osnovnu SA. Ključan faktor je bolja inicijalizacija (pohlepna ruta + 2-opt + or-opt + PackIterative) koja daje značajno bolju početnu tačku. Odvojena faza rute (pohlepna pretraga) i plana punjenja (SA bit-flip) osigurava fokusirano istraživanje svakog podprostora.

GAImproved takođe nadmašuje osnovnu GA, primarno zahvaljujući boljoj inicijalizaciji (greedy NN + 2-opt + PackIterative za sve jedinke) i EAX ukrštanju koje čuva dobre TSP ivice. Međutim, visoka računska cena lokalne pretrage po potomku ograničava broj iteracija.

GWO daje najlošije rezultate od svih algoritama, posebno na nekorelisanoj instanci. Razlozi su višestruki: (1) prevremena konvergencija čitavog čopora prema tri vođe, (2) delimične swap sekvence ne garantuju kvalitetne međurute, (3) glasanje većine za plan punjenja gubi individualne TTP-specifične adaptacije svakog vuka. Visoka standardna devijacija potvrđuje nestabilnost algoritma. Ovo sugerira da GWO u svom originalnom obliku nije prilagođen za kombinatorne probleme poput TTP-a bez značajnih modifikacija.

Opšti trendovi konzistentni su s literaturom [7]: algoritmi koji eksplicitno rade lokalnu pretragu nad oba podproblema (SA, GAImproved) ostvaruju bolje rezultate nego algoritmi koji zanemaruju jednu od dimenzija. Jako korelisane instance često daju više vrednosti jer predmeti imaju veće faktore profit/težina, ali su teže za KP rešavače.

7.4 Konvergencija

ACO i GAImproved pokazuju postupno poboljšanje kroz iteracije, dok SA ima karakteristični profil: visoku varijabilnost u početku (visoka temperatura) s postepenim stabiliziranjem. GWO konvergira brzo, ali na lošijem lokalnom optimumu.

8 Zaključak

U ovom radu prikazali smo matematičku formulaciju Problema putujućeg lopova, analizirali zašto nezavisno rešavanje podproblema nije optimalno, opisali standardni referentni skup instanci i implementirali šest metaheurističkih algoritama.

Ključni zaključci:

1. **Kvalitet inicijalizacije je presudan:** algoritmi koji ulažu u bolje početne tačke (pohlepna ruta + lokalna pretraga) konzistentno nadmašuju varijante sa nasumičnom inicijalizacijom. Ovo je konzistentno s nalazima Faulknera i saradnika [3] koji su pokazali da je S5 (strategija ponovnog pokretanja s više tura) iznenađujuće efikasan.
2. **Zavisno optimizovanje:** algoritmi koji istovremeno optimiziraju rutu i plan punjenja (SAImproved, GAIImproved) ostvaruju bolje rezultate od onih koji ih tretiraju odvojeno. Ovo potvrđuje centralnu tezu TTP-a [1, 4].
3. **GWO za TTP zahteva fundamentalne modifikacije:** originalni GWO nije pogodan za kombinatorne prostore. Naša diskretna adaptacija (swap sekvence + glasanje većine) poboljšava primenljivost, ali GWO i dalje zaostaje za specijalizovanim pristupima. Buduća istraživanja mogla bi ispitati hibridizaciju GWO-a s PackIterative lokalnom pretragom ili restart strategijama.
4. **Nijedan algoritam nije superioran na svim instancama:** konzistentno sa zaključcima studije algoritamskih paketa [7], različiti algoritmi pokazuju različite prednosti za različite tipove instanci.

Literatura

- [1] Bonyadi, M.R., Michalewicz, Z., Barone, L. (2013). *The Travelling Thief Problem: The First Step in the Transition from Theoretical Problems to Realistic Problems*. IEEE Congress on Evolutionary Computation (CEC), pp. 1037–1044.
- [2] Polyakovskiy, S., Bonyadi, M.R., Wagner, M., Michalewicz, Z., Neumann, F. (2014). *A Comprehensive Benchmark Set and Heuristics for the Traveling Thief Problem*. Genetic and Evolutionary Computation Conference (GECCO), pp. 477–484. ACM.
- [3] Faulkner, H., Polyakovskiy, S., Schultz, T., Wagner, M. (2015). *Approximate Approaches to the Traveling Thief Problem*. Genetic and Evolutionary Computation Conference (GECCO), pp. 385–392. ACM.
- [4] Mei, Y., Li, X., Yao, X. (2014). *On Investigation of Interdependence Between Sub-Problems of the Travelling Thief Problem*. Soft Computing, 20(1), pp. 157–172.
- [5] Wagner, M. (2016). *Stealing Items More Efficiently with Ants: A Swarm Intelligence Approach to the Travelling Thief Problem*. International Conference on Swarm Intelligence (ANTS).

- [6] Yafrani, M.E., Ahiod, B. (2016). *Population-Based vs. Single-Solution Heuristics for the Travelling Thief Problem*. Genetic and Evolutionary Computation Conference (GECCO). ACM.
- [7] Wagner, M., Lindauer, M., Misir, M., Nallaperuma, S., Hutter, F. (2016). *A Case Study of Algorithm Selection for the Traveling Thief Problem*. arXiv:1609.00260.
- [8] Mirjalili, S., Mirjalili, S.M., Lewis, A. (2014). *Grey Wolf Optimizer*. Advances in Engineering Software, 69, pp. 46–61.
- [9] Polyakovskiy, S., Neumann, F. (2015). *Packing While Traveling: Mixed Integer Programming for a Class of Nonlinear Knapsack Problems*. Integration of AI and OR Techniques in Constraint Programming, LNCS 9075, pp. 330–344. Springer.
- [10] Reinelt, G. (1991). *TSPLIB – A Traveling Salesman Problem Library*. ORSA Journal on Computing, 3(4), pp. 376–384.
- [11] Stützle, T., Hoos, H.H. (2000). *MAX–MIN Ant System*. Future Generation Computer Systems, 16(8), pp. 889–914.